

## Saturation Monitoring

This module contains saturation-monitoring definitions. These are used to monitor saturation in a GitLab instance.

More details can be found at <https://about.gitlab.com/handbook/engineering/infrastructure-platforms/capacity-planning/>.

## Capacity Planning

We use the saturation point definitions here as input for our capacity planning and forecasting tool Tamland.

### Saturation points

A saturation point refers to a particular saturation metric and its respective SLO definitions. In this context, a SLO is a named threshold for the given metric. Once the metric breaches this threshold or is predicted to breach it in the future, certain actions take place: We create capacity warning issues or trigger actual alerts.

**Definition** Currently, there are two types of SLOs:

1. **soft**: Once predicted to breach or actually breaching, we create capacity warning issues.
2. **hard**: Used for alerting the on-call if the threshold is breached and the respective service is tagged with severity **s2** or higher

Here is an example definition:

```
{
  cpu: resourceSaturationPoint({
    title: 'Average Service CPU Utilization',
    severity: 's3',
    horizontallyScalable: true,
    appliesTo: metricsCatalog.findVMProvisionedServices(first='gitaly'),
    description: |||
      This resource measures average CPU utilization across an all cores in a service fleet.
      If it is becoming saturated, it may indicate that the fleet needs
      horizontal or vertical scaling.
    |||,
    grafana_dashboard_uid: 'sat_cpu',
    resourceLabels: [],
    burnRatePeriod: '5m',
    query: |||
      1 - avg by (%(aggregationLabels)s) (
        rate(node_cpu_seconds_total{mode="idle", %(selector)s}[% (rangeInterval)s])
      )
    |||
  })
}
```

```

    |||,
    slos: {
      soft: 0.80,
      hard: 0.90,
    },
  }),
}

```

In this case, we can see the threshold for **soft** and **hard** SLO set to 80% and 90%, respectively.

**Forecasting parameters** A saturation point may also override default capacity planning parameters used for forecasting this saturation point.

```

{
  pg_int4_id: resourceSaturationPoint({
    title: 'Postgres int4 ID capacity',
    // ... truncated for brevity
    capacityPlanning: {
      strategy: 'quantile95_1w', // default: quantile95_1h
      forecast_days: 365,        // default: 90
      historical_days: 730,       // default: 360
    },
  }),
}

```

For a full overview of forecasting parameters, please refer to Tamland's documentation and reach out to Scalability::Projections for any questions.

### Adjusting SLOs and forecasting parameters

Soft SLO threshold and forecasting parameters should be tuned so we get capacity warnings at the right point in time.

### Component overrides

While a saturation point is an abstract definition used by a number of services, we can override parameters used in forecast models on a per-component basis. A component is a specific service at the given saturation point, for example we can look at the **pg\_int4\_id** saturation point for the **patroni-ci** service and adjust forecasting parameters for this individual component. This can be done in the service definition inside the metrics catalog.

**Overriding capacity planning parameters** By default, we forecast 90 days, consider a maximum of 1 year of history for the forecast model and base this on the **quantile95\_1h** metric. This can be changed for individual components.

Notice that in the case of component overrides, we specify parameters on the service definition. Below example shows how to override capacity planning parameters for the `memory` component for the `patroni` service.

```
# /metrics-catalog/services/patroni.jsonnet
```

```
metricsCatalog.serviceDefinition(  
  patroniRailsArchetype( /* ... */ )  
  +  
  {  
    capacityPlanning: {  
      components: [  
        {  
          name: 'memory',  
          parameters: {  
            forecast_days: 180,  
            historical_days: 500,  
            strategy: 'quantile95_1w',  
            changepoints: [  
              '2023-04-26',  
              '2023-04-28',  
            ],  
            ignore_outliers: [  
              {  
                start: '2023-03-01',  
                end: '2023-03-10'  
              }  
            ]  
          },  
        ],  
      },  
    ],  
  }  
})
```